

Fiche n°2: squelette de code C++ proposé dans l'environnement MBED

1.1. Squelette de code c++ proposé pour démarrer le projet

On fournit ci-dessous un squelette de programme permettant le démarrage du projet

Des zones de commentaires doivent vous aider à compléter vos morceaux de code au fur et à mesure de l'avancement du projet

```
//***** zone des includes de bibliothèque *****

#include <stdlib.h>
#include <string.h>
#include "mbed.h"
#include "rtos.h" // need for main thread sleep
#include "html.h" // need for html patch working with web server
#include "bloc_io.h"

//***** defines for geometrical parameters of the hub motor*****
#define RADIUS 0.2F // wheel size
#define NBPOLES 8 // magnetic pole number
#define DELTA_T 0.1F // speed measurement counting period

//***** instanciation or MyPLD object needed to communicate with PLD*****
Bloc_IO MyPLD(p25,p26,p5,p6,p7,p8,p9,p10,p23,p24); // instantiate object needed to
communicate with PLD
    // analog input connected to mbed
    // valid pmw mbed pin
Serial pc(USBTX, USBRX); // tx, rx
    // Top_Hall Pin

/***** persistent file parameters section *****/
LocalFileSystem local("local"); // Create the local filesystem under the
name "local"

/***** web server section *****/

var_field_t tab_balise[10]; //une balise est présente dans le squelette
int giCounter=0; // acces counting

/***** can bus section *****/
    // determine message ID used to send Gaz ref over can bus
#define _CAN_DEBUG // used to debug can bus activity
//#define USE_CAN_REF // uncomment to receive gaz ref over can_bus
CAN can_port (p30, p29); // initialisation du Bus CAN sur les broches 30 (rd) et 29(td)
for lpc1768 + mbed shield
bool bCan_Active=false;

//***** LPC1768 local hardware I/O initialisation *****/
DigitalOut led1(LED1); //initialisation des Leds présentes sur le micro-contrôleur
Mbed*/
DigitalOut led2(LED2);
DigitalOut led3(LED3); // blink when can message is sent
```

```
DigitalOut led4(LED4); // blink when can message is received
```

```
//***** local function prototypes *****
```

```

/***** Read persistent data from text file located on local file system
*****/
// put your read file fonction used to initialize the persistente variables

```

```

/***** write persitant data to text file located on local file system
*****/
//put your write file fonction used to save the persistente variables to text file

```

```

//***** calibration gaz function needed to record min_gaz and max_gaz value to
persistent text file *****

```

```
// *****top hall counting interrupt needed for speed measurement
```

```

//***** timer interrupt for speed measurement each 100ms
*****

```

```

//***** Timer Interrupt for gaz ref management each 10ms
*****

```

```

/***** main cgi function used to patch data to the web server thread
*****/
void CGI_Function(void) // cgi function that patch web data to empty web page
{ char ma_chaine4[20]={}; // needed to form html response

}

```

```
//***** CAN BUS SECTION *****
```

```

void CAN_REC_THREAD(void const *args)
{ int iCount,iError;

while (bCan_Active)
{Thread::wait(100); // wait 100ms
// code todo

}

}

```

```
//***** main function *****

int main() {
char cChoix=0;

//***** web section
*****/
//Init_Web_Server(&CGI_Function); // create and initialize tcp server socket and pass
function pointer to local CGI function
//Thread WebThread(Web_Server_Thread);// create and launch web server thread
//***** main cgi function used to patch data to the web server thread
*****/

//***** end web section
***** /

pc.printf(" programme scooter mbed \n");

//***** can bus section initialisation
*****/
//bCan_Active=true;// needed to lauchn CAN thread
//Thread CanThread(CAN_REC_THREAD);// create and launch can receiver thread
//***** end can bus section
*****/

while(cChoix!='q' and cChoix!='Q')
{pc.printf(" veuillez saisir un choix parmi la liste proposee: \n");
pc.printf(" a:saisie consigne pwm \n");
pc.printf(" q:quitter \n");

/***** multithreading : main thread need to sleep in order to allow web
response */
while (pc.readable()==0) // determine if char availabler
{Thread::wait(10);} // wait 10 until char available on serial input
/***** end of main thread sleep *****/

pc scanf(" %c",&cChoix);
switch (cChoix){
case 'a':
break;
case 'q':
break;
}
} // end while

//***** thread deinit *****
//DeInit_Web_Server();
//bCan_Active=false;
//CanThread=false;// close can received thread
pc.printf(" fin programme scooter mbed \n");
} // end main
```

1.2. Analyse de la structure du squelette de code

La première chose à remarquer est que le code que vous allez devoir écrire doit pouvoir exécuter plusieurs processus bloquant à la fois. Pour pouvoir traiter ces processus en temps partagé dans le processeur, il faut donc impérativement utiliser le noyau multitâche de l'environnement MBED (bibliothèque rtos.h)

C'est très simple : chaque processus bloquant doit être encapsulé d'un thread d'exécution séparé de la main() (c'est un peu comme si on avait plusieurs main() qui s'exécutaient en quasi parallélisme en temps processeur partagé). La main() est lui-même un thread (le thread de départ qui a la priorité maximale sur les autres threads).

Depuis la main(), on peut créer d'autres thread en les instanciant de la manière suivante :

- Supposons que nous avons une fonction de thread qui se nomme void CAN_REC_THREAD (void).
On reste dans cette fonction de thread tant qu'une variable booléenne « bCan_Active » globale sera true :
- Depuis la main, on instancie ce nouveau processus d'exécution (thread) via le constructeur suivant :
bCan_Active=true; // needed to launch CAN thread
Thread CanThread(CAN_REC_THREAD); // create and launch can receiver thread
- à la fin pour tuer le thread secondaire depuis la main il suffit de rendre le booléen de commande « bCan_Active »=false et le thread CAN_REC_THREAD() s'arrêtera et disparaîtra de lui-même.

Attention : pour que le noyau temps réel du Mbed puisse partager le temps du uC entre tous les threads, il faut impérativement que ces derniers passent en mode sommeil de temps en temps via la fonction Thread::wait(100); // wait 100ms

Dans cet exemple, le thread qui a appelé la fonction wait passe en sommeil pour 100ms ce qui permet de libérer le temps uC pour s'occuper d'un autre thread de priorité égale ou inférieure.

Veuillez donc à prévoir ces temps de mise en sommeil dans vos boucle d'attente de vos différents thread (la main() y compris).

1.3. Liste minimum des processus bloquant à gérer dans cette application

Il y a en tout 3 processus bloquant (thread) à gérer dans ce code que l'on ne peut pas traiter en mode d'interruption

Thread Main : attente de l'arrivée d'un caractère pour la sélection d'un choix dans le menu console

```
while(cChoix!='q' and cChoix!='Q')
{pc.printf(" veuillez saisir un choix parmi la liste proposee: \n");
pc.printf(" a:saisie consigne pwm \n");
pc.printf(" q:quitter \n");

/***** multithreading : main thread need to sleep in order to allow web
response */
while (pc.readable()==0) // determine if char available
{Thread::wait(10);} // wait 10 until char available on serial input
/***** end of main thread sleep *****/

pc scanf(" %c",&cChoix);
switch (cChoix){
case 'a':
break;
case 'q':
break;
}
} // end while
```

On remarque que l'attente d'un caractère (scanf) est bloquant pour la tâche de fond. Aussi afin de libérer le processeur pour d'autres thread, avant d'appeler la fonction scanf on vérifie si un caractère a été saisi ou pas par l'utilisateur via la fonction [pc.readable\(\)](#) qui n'est pas bloquante.

En fonction, on décide ou pas d'appeler le scanf pour récupérer le caractère saisi.

Notez que pendant l'attente d'un caractère, le thread courant (main) doit être mis en sommeil {Thread::wait(10);} afin de libérer le uC pour gérer les autres threads, le thread main() étant le plus prioritaire

Le thread de gestion de la communication can bus

Lors de l'avancement du projet, vous serez amené à préparer une fonction de réception ou d'émission de trames de données via le bus can. Là encore il n'y a pas de possibilité de déclencher une IT en réception ou fin d'émission de trame. On va donc attendre dans un thread séparé qu'une trame arrive

/***** CAN BUS Thread SECTION *****/

```
void CAN_REC_THREAD(void const *args)
{ int iCount,iError;

  while (bCan_Active)
  {Thread::wait(100);// wait 100ms
    // code todo

  }

}
```

Le thread de gestion de la création d'une page web dynamique

Là encore, notre application devra attendre de manière bloquante l'arrivée d'une requête web (protocole http) en provenance d'un navigateur client via le port Ethernet.

Pour cela, la bibliothèque qui permet la création de cette page html de réponse (html.cpp / html.h) possède un thread interne qui attends l'arrivée d'une requête http.

Pour nous depuis la main cela consiste simplement à instancier ce thread web

```
Init_Web_Server(&CGI_Function); // create and initialize tcp server socket and pass
function pointer to local CGI function
```

Cette fonction permet de vectoriser le thread vers la fonction qui permettra la mise à jour des variables dynamiques dans la page web à renvoyer au client (ici la fonction se nomme void CGI_Function(void))

Puis on instancie le thread du serveur web depuis la main() :

```
Thread WebThread(Web_Server_Thread);// create and launch web server thread
```

En fin de main(), afin de bloquer et éliminer ce thread, il suffira d'appeler la fonction DeInit_Web_Server(); qui se chargera de passer le booléen de contrôle du thread web à la valeur false